Studio Nx

P1-08 회원 정보 API — 함수 설명 합본 (35~43)

2026-10-07

목차

1. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/35_user_service_profile_image_url.md
2. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/36_user_service_to_me_response.md
3. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/37_user_service_update_me.md
4. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/38_user_service_save_profile_image.md
5. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/39_user_service_delete_profile_image.md
6. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/40_user_service_profile_image_file.md
7. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/41_users_route_me.md
8. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/42_users_route_profile_image.md
9. ai-notes/backend(fastAPI_Python)/P1-08 - 회원 정보 API/43_users_route_profile_image_file.md

Backend(FastAPI/Python) 35 — `profile_image_url()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보 — 프로필 사진 주소 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	34 `POST /auth/logout` · 36 `to_me_response()`

1. 하는 일

회원의 프로필 사진 **저장 경로**를 화면이 `<img src>` 에 넣을 **주소**로 바꾼다. 사진이 없으면 `None`.

`user.profile_image_path` (DB)	돌려주는 값
`None` · `""`	`None` → 화면은 닉네임 첫 글자를 보여 줌
`"profiles/3f9c2b7e0d1a4c5b8e6f7a9b0c1d2e3f.jpg"`	`"/api/v1/profile-images/3f9c2b7e0d1a4c5b8e6f7a9b0c1d2e3f.jpg"`

2. 언제 쓰나

쓰는 곳	하는 일
36 `to_me_response()`	`GET /users/me` 등의 응답 `profile_image_url` 칸
(P1-16) 대시보드 응답	프로필 카드 사진

사용자 결정 (2026-10-07) — 주소에 회원 번호를 넣지 않음

템플릿은 `/api/v1/users/{회원 번호}/profile-image?v=…` 였다. 이 주소는 로그인 없이 열린다(`<img>` 는 토큰을 붙일 수 없음). 그러면 누구나 번호를 1, 2, 3… 바꿔 가며 **그 번호의 회원이 있는지 · 사진이 무엇인지** 볼 수 있다. P1-04 때 정한 **탈퇴 추적 방지 ③ "회원 번호를 받는 공개 API 는 만들지 않음"** 과 어긋나서, 사용자가 **무작위 파일 이름**을 골랐다.

	템플릿	결정
저장 경로	`profiles/{회원 번호}/{무작위}.jpg`	`profiles/{무작위 32자}.jpg`
주소	`/api/v1/users/85/profile-image?v=1696651200`	`/api/v1/profile-images/{무작위 32자}.jpg`
남의 사진 찾아보기	번호만 바꾸면 됨	32자(16진수 = 2^{128} 가지)를 맞혀야 함
사진을 바꿨을 때 캐시	`?v=` 로 해결	이름 자체가 바뀌어 필요 없음

함께 고친 문서 · 주석: PRD 01 §7 의 API 표, 이 파일의 사진 저장(38) · 사진 파일(40) 템플릿 주석, `users.py` 머리말. 라우터 주소 자체는 43 을 구현할 때 바꾼다.

3. 코드

import (파일 맨 위)

```
from pathlib import Path, PurePosixPath

from fastapi import UploadFile
from sqlalchemy.orm import Session

from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
```

`PurePosixPath` 를 이번에 추가했다.

상수 (이번에 추가)

```
# 사진을 두는 폴더 (settings.storage_dir 아래) · 화면이 사진을 받는 주소 앞부분
PROFILE_DIR = "profiles"
PROFILE_IMAGE_URL = "/api/v1/profile-images/"
```

함수

```
def profile_image_url(user: User) -> str | None:
    if not user.profile_image_path:
        return None
    return PROFILE_IMAGE_URL + PurePosixPath(user.profile_image_path).name
```

문법 풀이

import 문법	뜻
`from pathlib import Path, PurePosixPath`	`Path` 는 실제 파일(디스크)용, `PurePosixPath` 는 **글자만** 다루는 경로 — 디스크를 보지 않고, 구분자는 늘 `/`

식	뜻
`-> str | None`	글자 또는 `None` 을 돌려줌 (Python 3.10 문법)
`if not user.profile_image_path:`	`None` 과 빈 글자 `""` 를 함께 걸러 냄
`PurePosixPath("profiles/3f9c….jpg").name`	마지막 부분 = 파일 이름 `"3f9c….jpg"`
`PROFILE_IMAGE_URL + …`	글자 잇기

4. 흐름

1. 저장 경로가 없으면 `None`.
2. 있으면 파일 이름만 꺼내 주소 앞부분에 붙여 돌려준다.

5. 왜 이렇게

선택	이유
파일 이름만 주소에 (폴더 빼고)	폴더 이름(`profiles`)은 서버 안의 사정이다. 주소는 `/profile-images/` 로 이미 사진임을 말한다
`PurePosixPath` (`Path` 아님)	DB 의 경로는 늘 `/` 로 저장한다(DB.md §1.5 상대 경로). Windows 에서 `Path` 를 쓰면 `\` 구분자로 바뀔 수 있다. 디스크를 볼 필요도 없다

선택	이유
문자열 자르기(`split('/')[-1]`) 대신 `PurePosixPath`	뜻이 이름에 드러난다
주소 앞부분을 상수로	40 · 43(사진 파일 라우터)이 같은 주소를 쓴다. 한 곳만 고치면 된다
DB 를 보지 않음	이미 가진 `User` 만으로 계산 — 응답을 만들 때마다 부르므로 가볍게

6. 시험 (`tests/test_user_service.py` 새 파일 · 5개)

시험	확인한 것
사진 없음 (`None`) · 빈 글자	`None`
`profiles/{32자}.jpg` · `.webp`	`/api/v1/profile-images/{32자}.jpg` · `.webp`
회원 번호 85	주소에 `85` · `/users/` 가 없음

전체 시험 239개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 36 — `to_me_response()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보 — 내 정보 응답 만들기 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	35 `profile_image_url()` · 37 `update_me()`

1. 하는 일

DB 의 회원 한 줄(`User`)을 화면에 보낼 **내 정보 응답(`MeResponse`)** 으로 바꾼다. 정해 둔 7칸만 담는다.

```
{
  "user_id": 85,
  "email": "demo@example.com",
  "nickname": "demo_user",
  "role": "USER",
  "profile_image_url": "/api/v1/profile-images/3f9c….jpg",
  "created_at": "2026-10-07T01:12:30",
  "last_login_at": "2026-10-07T09:30:00"
}
```

담지 않는 칸 (User 에는 있음)	이유
`password_hash`	해시라도 밖으로 나가면 안 됨 — 무작위 대입으로 원래 비밀번호를 찾는 데 쓰일 수 있음
`status` · `deleted_at`	이 응답은 로그인한 **활동 회원**만 받음 (06 이 이미 막음)
`privacy_agreed_at` · `privacy_version` · `storage_*` 등	회원 정보 화면이 쓰지 않음. 용량은 P1-16 대시보드 응답이 따로 줌

2. 언제 쓰나

쓰는 곳 (P1-08 라우터)	
`GET /users/me` (41)	내 정보
`PATCH /users/me` (41)	닉네임을 바꾼 뒤 바뀐 정보
`POST · DELETE /users/me/profile-image` (42)	사진을 바꾸거나 지운 뒤 바뀐 정보 (새 `profile_image_url`)

화면: `userinfo.js` `loadMe`(P1-10), `mypage.js`.

`user_id` 를 넣는 것은 괜찮다 — 이 응답은 **토큰을 가진 본인**에게만 간다. 탈퇴 추적 방지 ③ 은 "남의 번호로 조회하는 공개 API" 를 막는 것이다.

3. 코드

import (파일 맨 위 — 35 와 같음, 추가 없음)

```
from pathlib import Path, PurePosixPath

from fastapi import UploadFile
from sqlalchemy.orm import Session

from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
```

이 함수는 **`User`** · **`MeResponse`** 와 35 `profile_image_url` 을 쓴다.

함수

```
def to_me_response(user: User) -> MeResponse:
    return MeResponse(
        user_id=user.user_id,
        email=user.email,
        nickname=user.nickname,
        role=user.role,
        profile_image_url=profile_image_url(user),
        created_at=user.created_at,
        last_login_at=user.last_login_at,
    )
```

문법 풀이

import 문법	뜻
`from app.schemas.user import MeResponse`	템플릿에 있던 응답 모양 (pydantic `BaseModel`)

식	뜻
`MeResponse(user_id=…, …)`	이름 붙인 인자(키워드 인자)로 칸마다 값을 넣어 만듦. 빠진 칸 · 틀린 형식이면 pydantic 이 바로 오류
`role=user.role`	DB 는 `UserRole.USER`(StrEnum). `StrEnum` 은 글자이기도 해서 응답 칸 `str` 에 그대로 들어감 → JSON 에 `"USER"`
`created_at` (`datetime`)	FastAPI 가 JSON 으로 바꿀 때 `"2026-10-07T01:12:30"` 모양 글자로

4. 흐름

1. 회원의 칸 6개를 그대로 옮긴다.
2. 사진은 35 로 주소를 만든다.
3. 응답 객체를 돌려준다.

5. 왜 이렇게

선택	이유
칸을 **하나씩** 적음	`MeResponse.model_validate(user)`(ORM 에서 자동으로 꺼내기)도 되지만, 그러려면 스키마에 `from_attributes` 설정이 필요하고 `profile_image_url` 은 DB 칸이 아니라 따로 계산해야 한다. 적어 두면 무엇이 나가는지 한눈에 보인다
응답 칸을 스키마로 고정	나중에 `User` 에 칸이 늘어도 응답에 저절로 섞이지 않는다 — 시험이 "칸은 7개뿐" 을 확인한다

선택	이유
DB 를 다시 읽지 않음	부르는 쪽이 이미 최신 `User` 를 가지고 있다 (수정 뒤에는 37 · 38 이 `refresh`)

6. 시험 (`tests/test_user_service.py` 에 3개 추가 · 모두 8개, 실제 MySQL)

시험	확인한 것
새 회원	7칸 값이 DB 와 같음, `role` `"USER"`, 사진 · 마지막 로그인 `None`, `created_at` 은 DB 가 채운 값
응답 칸	정확히 7개. JSON 안에 비밀번호 해시 글자 · `password` 가 없음
관리자 · 사진 · 로그인 기록	`role` `"ADMIN"`, `last_login_at` 그대로, 사진 주소 `/api/v1/profile-images/{32자}.png`

전체 시험 242개 통과, `ruff` 검사 통과. 시험에서 만든 회원은 시험이 끝나면 되돌려진다 (conftest 의 savepoint).

Backend(FastAPI/Python) 37 — `update_me()` (+ `MeUpdateRequest`)

항목	내용
파일	`studionx/backend/app/services/user_service.py` · `app/schemas/user.py`
작업	P1-08 회원 정보 — 닉네임 변경 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	36 `to_me_response()` · 38 `save_profile_image()`

1. 하는 일

내 닉네임을 바꾼다. 가입 때와 같은 규칙(2~20자, 다른 사람과 겹치지 않음, 탈퇴 3일 규칙)을 지킨다. 이메일은 바꿀 수 없다(FR-106).

지금 닉네임 → 보낸 값	결과
`my_name` → (칸 없음) · `null` · `"my_name"`	아무것도 안 함
`old_name` → `" New_Name "`	`New_Name` 으로 저장 (앞뒤 공백 제거)
`Demo_Case` → `"demo_case"` (대소문자만)	저장 — 내 닉네임이므로
`my_name` → `"taken_name"` (남이 `Taken_Name` 사용 중)	409 `NICKNAME_TAKEN` "This nickname is already in use."
`my_name` → `"gone_nick"` (어제 탈퇴한 회원의 닉네임)	409 `REJOIN_TOO_SOON` "This nickname can be used again after Oct 9, 2026 (UTC)."
`my_name` → `"free_again"` (4일 전 탈퇴한 회원의 닉네임)	저장. 탈퇴 회원 쪽은 `deleted_user_0000000085` 로 비켜 둠 (P1-04 `release_withdrawn`)
`" a "` · 21자	422 — 스키마에서 막힘 (아래)

2. 언제 쓰나

쓰는 곳	
`PATCH /users/me` (41)	본문 `{"nickname": "…"}` → 이 함수 → 36 으로 응답
화면	userInfo.html 의 닉네임 칸 · Check 버튼 · Save changes (P1-10)

템플릿에서 바꾼 것 2가지

	템플릿	바꾼 것	이유
스키마 `MeUpdateRequest.nickname`	`str` + 길이 2~20	가입과 같은 **`Nickname`** 타입	템플릿대로면 `" a "` 가 5자로 통과하고, 저장할 때 공백을 지우면 1자 닉네임이 생긴다. `Nickname` 은 공백을 먼저 지우고 길이를 잰다
"지금 닉네임과 다르면 확인"	글자가 다르면 확인	**DB 가 같다고 보면** 확인을 건너뜀	DB 비교 규칙(utf8mb4_0900_ai_ci)은 대소문자를 구분하지 않는다. `Demo_Case` → `demo_case` 를 확인하면 **내 닉네임**이 "사용 중" 으로 나와 바꿀 수 없다

3. 코드

import (파일 맨 위)

```
from pathlib import Path, PurePosixPath

from fastapi import UploadFile
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
from app.services.auth_service import check_nickname
```

`select` · `IntegrityError` · `AppError` · `ErrorCode` · `check_nickname` 을 이번에 추가했다.

스키마 (`schemas/user.py`)

```
from app.schemas.auth import Nickname

class MeUpdateRequest(BaseModel):
    nickname: Nickname | None = None
```

함수

```
def update_me(db: Session, user: User, data: MeUpdateRequest) -> User:
    nickname = data.model_dump(exclude_unset=True).get("nickname")
    if nickname is None or nickname == user.nickname:
        return user

    mine = db.scalar(
        select(User.user_id).where(User.user_id == user.user_id, User.nickname == nickname)
    )
    if mine is None:
        result = check_nickname(db, nickname, release=True)
        if result.reason == "REJOIN_TOO_SOON":
            when = result.available_at
            raise AppError(
                ErrorCode.REJOIN_TOO_SOON,
                409,
                f"This nickname can be used again after {when:%b} {when.day}, {when.year} (UTC).",
                details={"nickname": "REJOIN_TOO_SOON", "available_at": when.isoformat()},
            )
        if not result.available:
            raise AppError(ErrorCode.NICKNAME_TAKEN, 409, "This nickname is already in use.")

    user.nickname = nickname
    try:
        db.commit()
    except IntegrityError:
        # 확인과 저장 사이에 다른 사람이 같은 닉네임으로 먼저 바꾼 경우
        db.rollback()
        raise AppError(ErrorCode.NICKNAME_TAKEN, 409, "This nickname is already in use.") from None
```

```
    db.refresh(user)
    return user
```

문법 풀이

import 문법	뜻
`from app.services.auth_service import check_nickname`	P1-04 (19) 의 닉네임 확인을 그대로 씀
`from sqlalchemy.exc import IntegrityError`	UNIQUE 등 DB 규칙 위반 오류

식	뜻
`Nickname | None = None`	닉네임 타입 또는 `None`, 안 보내면 `None`
`data.model_dump(exclude_unset=True)`	**보낸 칸만** 사전으로 — 보내지 않은 칸은 빠짐 (나중에 칸이 늘어도 안 보낸 칸은 안 바뀜)
`.get("nickname")`	없으면 `None`
`select(User.user_id).where(A, B)`	`WHERE A AND B` — 그 회원이면서 닉네임이 DB 규칙상 같은 행
`db.scalar(…)`	첫 행의 첫 값, 없으면 `None`
`f"{when:%b}"`	날짜를 `Oct` 처럼 달 이름으로
`raise … from None`	원래 DB 오류(IntegrityError)를 오류 기록에 붙이지 않음 — 예상한 상황이라서

4. 흐름

1. 보낸 닉네임을 꺼낸다. 없거나 지금과 똑같으면 그대로 돌려준다.
2. DB 가 "내 닉네임과 같다" 고 보는지 확인 (대소문자 · 악센트만 다른 경우).
3. 아니면 P1-04 의 닉네임 확인 → 탈퇴 3일 안이면 409 `REJOIN_TOO_SOON`, 사용 중이면 409 `NICKNAME_TAKEN`. (3일 지난 탈퇴 닉네임은 이 안에서 비켜 둠)
4. 저장. 그사이 남이 먼저 가져갔으면(UNIQUE 위반) 되돌리고 409.
5. DB 에서 다시 읽어 돌려준다.

5. 왜 이렇게

선택	이유
똑같으면 DB 를 건드리지 않음	화면이 닉네임을 안 바꾸고 Save 를 눌러도 쓸데없는 쓰기 · 확인이 없다
`null` 도 "안 바꿈"	닉네임을 비울 수는 없다 (DB `NOT NULL`). 오류보다 무시가 안전
탈퇴 3일 문구를 따로 만듦	P1-04 의 `unavailable_error` 는 "You can **sign up** with this nickname again after…" — 가입 문구다. 이미 검토가 끝난 함수라 고치지 않고 여기서 수정 화면에 맞는 문구를 만들었다
`release=True`	3일 지난 탈퇴 닉네임은 가입처럼 바로 쓸 수 있어야 한다 (FR-108). 비켜 두기는 같은 `commit` 으로 함께 저장되고, 409 로 끝나면 저장하지 않아 취소된다
`IntegrityError` → 409	P1-04 가입과 같은 이유 — 확인과 저장 사이의 짧은 틈
마지막에 `refresh`	DB 가 바꾼 값(`updated_at`)까지 받아 36 응답에 씀

6. 시험 (`tests/test_user_service.py` 에 14개 추가 · 모두 22개, 실제 MySQL)

시험	확인한 것
스키마	`" ab "` → `"ab"`, 20자 통과 / `" a "` · 21자 · 빈 글자 → 오류
바꿀 것 없음 (칸 없음 · `null` · 같음)	같은 객체를 돌려주고 DB 그대로
새 닉네임 `" New_Name "`	DB 에 `New_Name`
대소문자만 (`Demo_Case` → `demo_case`)	저장됨
남의 닉네임 (대소문자만 다름)	409 `NICKNAME_TAKEN`, 내 닉네임 그대로
어제 탈퇴한 회원의 닉네임	409 `REJOIN_TOO_SOON`, 문구 · `details`, 내 닉네임 그대로
4일 전 탈퇴한 회원의 닉네임	저장됨, 탈퇴 회원은 `deleted_user_{번호 10자리}`
동시에 바꾸기 (확인을 통과시켜 흉내)	`commit` 의 UNIQUE 위반 → 409, 내 닉네임 그대로

전체 시험 256개 통과, `ruff` 검사 통과. 시험이 끝난 뒤 DB 에는 데모 계정과 사용자 계정만 남음.

Backend(FastAPI/Python) 38 — `save_profile_image()` (+ `image_extension()`)

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보 — 프로필 사진 올리기 · PRD 01 FR-106 (jpg/png/webp, 2MB)
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	37 `update_me()` · 39 `delete_profile_image()`

1. 하는 일

올라온 사진을 검사하고 `storage/profiles/{무작위 32자}.{확장자}` 에 저장한 뒤, 회원의 `profile_image_path` 를 바꾼다. 예전 사진 파일은 지운다.

올라온 파일	결과
JPG (1.2MB)	저장 → `profiles/3f9c….jpg`, 응답 사진 주소도 바뀜
브라우저는 `image/png` 라고 했지만 내용은 JPG	저장 — 확장자는 **내용대로** `.jpg`
GIF	415 `UNSUPPORTED_FILE_TYPE` "Use a JPG, PNG or WebP image."
`image/png` 라면서 내용은 HTML · 빈 파일	415 (같음)
2MB 를 1바이트라도 넘음	413 `FILE_TOO_LARGE` "Images must be 2 MB or smaller."
정확히 2MB	저장
이미 사진이 있음	새 사진 저장 → 예전 파일 삭제

오류일 때는 파일이 하나도 남지 않고, DB 도 그대로다.

2. 언제 쓰나

쓰는 곳	
`POST /users/me/profile-image` (42)	`multipart/form-data` 의 `file` 칸 → 이 함수 → 36 응답
화면	userInfo.html 의 Upload photo (P1-10) — 화면도 같은 종류 · 크기를 먼저 검사함 (`PHOTO_TYPES` · `PHOTO_MAX_BYTES`)

용량: 프로필 사진은 저장 한도 5GB 사용량에 **넣지 않음** (2026-10-07 사용자 결정) → `storage_used_bytes` 를 건드리지 않는다.

3. 코드

import (파일 맨 위)

```
from pathlib import Path, PurePosixPath
from uuid import uuid4
```

```
from fastapi import UploadFile
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
from app.services.auth_service import check_nickname
```

`uuid4` · `get_settings` 를 이번에 추가했다.

도우미 — 내용으로 종류 알아내기

```
def image_extension(data: bytes) -> str | None:
    if data.startswith(b"\xff\xd8\xff"):
        return ".jpg"
    if data.startswith(b"\x89PNG\r\n\x1a\n"):
        return ".png"
    if data[:4] == b"RIFF" and data[8:12] == b"WEBP":
        return ".webp"
    return None
```

함수

```
async def save_profile_image(db: Session, user: User, file: UploadFile) -> User:
    if file.content_type not in PHOTO_TYPES:
        raise AppError(ErrorCode.UNSUPPORTED_FILE_TYPE, 415, "Use a JPG, PNG or WebP image.")
    data = await file.read(PHOTO_MAX_BYTES + 1)
    if len(data) > PHOTO_MAX_BYTES:
        raise AppError(ErrorCode.FILE_TOO_LARGE, 413, "Images must be 2 MB or smaller.")
    extension = image_extension(data)
    if extension is None:
        raise AppError(ErrorCode.UNSUPPORTED_FILE_TYPE, 415, "Use a JPG, PNG or WebP image.")

    storage_dir = get_settings().storage_dir
    relative = f"{PROFILE_DIR}/{uuid4().hex}{extension}"
    new_file = storage_dir / relative
    new_file.parent.mkdir(parents=True, exist_ok=True)
    new_file.write_bytes(data)

    old_relative = user.profile_image_path
    user.profile_image_path = relative
    try:
        db.commit()
    except Exception:
        db.rollback()
        new_file.unlink(missing_ok=True)
        raise
    if old_relative:
        (storage_dir / old_relative).unlink(missing_ok=True)
    db.refresh(user)
    return user
```

문법 풀이

import 문법	뜻
`from uuid import uuid4`	무작위 고유 번호. `.hex` = 16진수 32자
`from app.core.config import get_settings`	설정(`.env`) — `storage_dir` = `studionx/storage` (git 에서 제외된 폴더)

식	뜻
`async def` · `await file.read(n)`	파일을 **최대 n 바이트**만 읽음. 업로드는 네트워크를 기다리므로 FastAPI 의 `UploadFile` 이 비동기
`b"\xff\xd8\xff"`	바이트 글자 — `\xff` = 16진수 FF 한 바이트
`data.startswith(…)` · `data[8:12]`	앞부분이 같은지 · 8 ~ 11번째 바이트
`storage_dir / relative`	`Path` 의 `/` = 경로 잇기
`mkdir(parents=True, exist_ok=True)`	중간 폴더까지 만들고, 이미 있어도 오류 없음
`unlink(missing_ok=True)`	파일 삭제, 없어도 오류 없음
`except Exception: … raise`	정리만 하고 **같은 오류를 다시** 던짐 (삼키지 않음)

4. 흐름

1. 브라우저가 알려 준 종류 확인 → 아니면 415.
2. 2MB + 1바이트까지만 읽기 → 넘으면 413.
3. 앞 바이트로 실제 종류 확인 → 아니면 415. 확장자는 여기서 정함.
4. 무작위 이름으로 파일 쓰기.
5. DB 에 새 경로 저장 → 실패하면 새 파일을 지우고 오류 그대로.
6. 성공했으면 예전 사진 파일을 지우고, DB 에서 다시 읽어 돌려줌.

5. 왜 이렇게

선택	이유
종류를 **두 번** 확인 (알려 준 값 + 내용)	`content_type` 은 브라우저(또는 공격자)가 마음대로 보낼 수 있다. HTML 을 `image/png` 라고 보내 저장시키면, 나중에 그 주소를 연 사람의 브라우저에서 스크립트가 돌 위험이 있다
확장자를 **내용**으로 정함	파일을 돌려줄 때(40 · 43) 확장자로 종류를 정한다. 내용과 확장자가 맞아야 한다
`PHOTO_MAX_BYTES + 1` 만 읽음	10GB 파일을 올려도 2MB 남짓만 메모리에 올라온다. 1바이트를 더 읽어야 "딱 2MB" 와 "넘음" 을 구분할 수 있다
저장 → DB → 예전 파일 삭제 순서 (템플릿은 "예전 파일을 지우고 → 저장")	DB 저장이 실패해도 예전 사진이 남는다. 템플릿 순서면 실패할 때 사진이 사라진다. 시험으로 확인
이름은 무작위 32자 (회원 번호 없음)	35 의 사용자 결정. 같은 회원이 사진을 바꾸면 이름도 바뀌어 캐시 문제도 없다
`except Exception` (넓게)	DB 연결이 끊기는 등 어떤 오류든 새 파일을 남기지 않으려고. 오류는 다시 던져 06 의 500 처리로 간다
그림 전체를 열어 보지는 않음	앞 바이트 검사로 종류는 확실히 가린다. 이미지 라이브러리(Pillow)로 다시 저장하면 더 안전하지만 패키지가 하나 는다 — 필요하면 나중에

선택	이유
사진 파일 응답에 `nosniff`	43 에서 `X-Content-Type-Options: nosniff` 를 붙여, 브라우저가 내용을 보고 종류를 바꿔 추측하지 않게 할 예정

6. 시험 (`tests/test_user_service.py` 에 17개 추가 · 모두 39개, 실제 MySQL)

사진은 시험용 임시 폴더(`tmp_path`)에 저장 — 진짜 `storage/` 는 건드리지 않음.

시험	확인한 것
`image_extension` 7가지	jpg · png · webp / gif · HTML · WAV(RIFF 지만 WEBP 아님) · 빈 파일 → `None`
JPG 저장	경로 `profiles/{32자}.jpg`, 파일 내용 같음, 파일 1개, 응답 주소, DB 경로, `storage_used_bytes` 0 그대로
png 라면서 jpg	`.jpg`
GIF · HTML · 빈 파일 · 2MB 초과	415 · 415 · 415 · 413, 파일 0개, 경로 그대로
정확히 2MB	저장
사진 바꾸기	새 이름, 예전 파일 삭제, 파일 1개
옛 파일이 이미 없음	오류 없이 저장
DB 저장 실패	새 파일 삭제, 예전 사진 파일 · 내용 그대로, 오류 그대로

전체 시험 273개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 39 — `delete_profile_image()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보 — 프로필 사진 지우기 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	38 `save_profile_image()` · 40 `profile_image_file()`

1. 하는 일

내 프로필 사진을 지운다. DB 의 `profile_image_path` 를 비우고, 저장된 파일도 지운다. 화면은 닉네임 첫 글자 동그라미로 돌아간다.

상황	결과
사진 있음	경로 `None`, 파일 삭제, 응답 사진 주소 `null`
사진 없음 · 두 번 누름	오류 없이 그대로 `None`
DB 에는 경로가 있는데 파일은 이미 없음	오류 없이 경로 `None`
DB 저장 실패	오류 그대로, **파일은 남김**

2. 언제 쓰나

쓰는 곳	
`DELETE /users/me/profile-image` (42)	userInfo.html 의 Remove 버튼 (P1-10) → 이 함수 → 36 응답
P1-09 회원 탈퇴 `withdraw()`	템플릿 6번 "사진 파일 지우기 (delete_profile_image 재사용, commit 은 마지막에 한 번)". 탈퇴 상태를 바꾼 **뒤 마지막에** 이 함수를 부르면 이 안의 `commit` 이 탈퇴 상태와 사진 지우기를 **함께** 저장한다

3. 코드

import (파일 맨 위 — 38 과 같음, 추가 없음). 이 함수는 `get_settings` 를 쓴다.

함수

```
def delete_profile_image(db: Session, user: User) -> User:
    old_relative = user.profile_image_path
    user.profile_image_path = None
    db.commit()
    if old_relative:
        (get_settings().storage_dir / old_relative).unlink(missing_ok=True)
    db.refresh(user)
    return user
```

문법 풀이

식	뜻
`old_relative = user.profile_image_path`	지우기 전에 경로를 따로 기억 — 바로 아래에서 `None` 으로 바꾸므로
`get_settings().storage_dir / old_relative`	실제 파일 위치 (`studionx/storage/profiles/….jpg`)
`unlink(missing_ok=True)`	파일 삭제, 이미 없어도 오류 없음
`db.refresh(user)`	DB 가 바꾼 `updated_at` 까지 다시 읽음

4. 흐름

1. 지금 경로를 기억하고 DB 의 경로를 비운다.
2. 저장(commit). 실패하면 여기서 오류 — 파일은 그대로.
3. 성공했고 예전 경로가 있었으면 파일을 지운다.
4. 다시 읽어 돌려준다.

5. 왜 이렇게

선택	이유
DB 먼저, 파일은 나중 (템플릿은 "파일 삭제 → DB")	38 과 같은 이유. 템플릿 순서로 하면 DB 저장이 실패할 때 DB 는 사진이 있다고 하는데 파일은 없어 사진이 깨진다. 이 순서면 실패해도 사진이 그대로 보인다 (반대로 파일 삭제가 실패하면 파일만 남음 — 화면에는 영향 없음)
오류 없이 끝나는 경우를 넓게	Remove 를 빠르게 두 번 누르거나, 파일을 손으로 지운 경우에도 사용자에게 오류를 보일 이유가 없다
`commit` 을 안에서 함 (매개변수로 끄지 않음)	템플릿은 탈퇴에서 "commit 은 마지막에 한 번" 이라고 했다. 탈퇴가 이 함수를 **맨 마지막에** 부르면 같은 결과가 된다. 켜고 끄는 매개변수를 두면 "파일을 언제 지우나" 가 부르는 쪽 책임이 되어 더 복잡해진다
38 의 `rollback` 같은 정리 코드가 없음	지울 새 파일이 없다. 오류가 나면 요청이 끝날 때 세션이 닫히며 되돌려진다

6. 시험 (`tests/test_user_service.py` 에 5개 추가 · 모두 44개, 실제 MySQL + 임시 폴더)

시험	확인한 것
사진 지우기	같은 객체를 돌려줌, 경로 · 응답 주소 · DB 모두 `None`, 파일 삭제, 폴더 비었음
사진 없음 · 두 번	오류 없음
파일이 이미 없음	오류 없이 경로 `None`
다른 회원의 사진	지워지지 않음
DB 저장 실패	오류 그대로, 파일 · 내용 그대로

전체 시험 278개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 40 — `profile_image_file()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보 — 사진 파일 찾기 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	39 `delete_profile_image()` · 41 `GET · PATCH /users/me`

1. 하는 일

주소의 사진 이름(`3f9c….jpg`)으로 **디스크의 실제 파일 경로**를 찾아 돌려준다. 보여 주면 안 되는 경우는 모두 같은 404.

`name`	결과
활동 회원의 사진 이름	`C:/…/studionx/storage/profiles/3f9c….jpg`
정지(`SUSPENDED`) 회원의 사진	경로 (보임)
탈퇴(`DELETED`) 회원의 사진 — 3일 안이어도	404
아무도 쓰지 않는 이름	404
DB 에는 있는데 디스크에 파일이 없음	404
`../../.env` · 대문자 · `.gif` · `….jpg.exe` · 31자 · 빈 글자	404 — **DB 를 보지 않고**

404 는 모두 `NOT_FOUND` "Image not found." — 어느 경우인지 알려 주지 않는다.

2. 언제 쓰나

쓰는 곳	
`GET /api/v1/profile-images/{name}` (43)	화면의 `<img src>` 가 부름 — 로그인 없이 (`<img>` 는 토큰을 붙일 수 없음). 이 함수가 준 경로를 `FileResponse` 로 보냄

35 의 사용자 결정대로, 회원 번호가 아니라 **올릴 때 만든 무작위 이름**으로 찾는다.

3. 코드

import (파일 맨 위)

```
import re
from pathlib import Path, PurePosixPath
from uuid import uuid4

from fastapi import UploadFile
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session
```

```
from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
from app.models import User
from app.models.user import UserStatus
from app.schemas.user import MeResponse, MeUpdateRequest
from app.services.auth_service import check_nickname
```

`re` · `UserStatus` 를 이번에 추가했다.

상수 (이번에 추가)

```
# 주소로 받는 사진 이름 모양 — 올릴 때 만드는 uuid4().hex(소문자 16진수 32자) + 허용 확장자
PHOTO_NAME = re.compile(r"[0-9a-f]{32}\.(jpg|png|webp)")
```

함수

```
def profile_image_file(db: Session, name: str) -> Path:
    not_found = AppError(ErrorCode.NOT_FOUND, 404, "Image not found.")
    if not PHOTO_NAME.fullmatch(name):
        raise not_found
    user = db.scalar(select(User).where(User.profile_image_path == f"{PROFILE_DIR}/{name}"))
    if user is None or user.status == UserStatus.DELETED:
        raise not_found
    storage_dir = get_settings().storage_dir.resolve()
    path = (storage_dir / user.profile_image_path).resolve()
    if not path.is_relative_to(storage_dir) or not path.is_file():
        raise not_found
    return path
```

문법 풀이

import 문법	뜻
`import re`	정규식(글자 모양 검사)
`from app.models.user import UserStatus`	회원 상태 목록 (P1-03 08)

식	뜻
`re.compile(r"…")`	정규식을 미리 한 번 만들어 둠. `r"…"` = `\` 를 그대로 쓰는 글자
`[0-9a-f]{32}` · `\.` · `(jpg|png|webp)`	소문자 16진수 32개 · 점 · 셋 중 하나
`PHOTO_NAME.fullmatch(name)`	**처음부터 끝까지** 모양이 맞아야 참 (`….jpg.exe` 는 거짓)
`.resolve()`	`..` · 바로 가기(심볼릭 링크)를 풀어 진짜 절대 경로로
`path.is_relative_to(storage_dir)`	경로가 그 폴더 **안**인지 (Python 3.9+)
`path.is_file()`	디스크에 파일로 있는지

4. 흐름

1. 이름 모양이 아니면 404 (DB 를 보지 않음).
2. 그 이름을 쓰는 회원을 찾는다. 없거나 탈퇴했으면 404.
3. 실제 경로를 만들고, storage 폴더 안인지 · 파일이 있는지 확인. 아니면 404.
4. 경로를 돌려준다.

5. 왜 이렇게

선택	이유
이름 모양 검사를 **맨 앞**에	`../../.env` 같은 이름은 DB 를 보기도 전에 끝난다. 모양이 맞는 이름만 경로가 될 수 있어서, 폴더 밖으로 나가는 길이 처음부터 막힌다
그래도 `is_relative_to` 를 한 번 더	템플릿 요구. DB 값이 어떤 이유로든 이상해져도(손으로 고침 등) storage 밖 파일을 보내지 않는다 — 이중 잠금
탈퇴는 3일 안이어도 404	탈퇴 = "사이트에서 나를 지움". 파일 자체는 P1-09 탈퇴가 39 로 지우지만, 그 전에도 보이지 않게 한다. 추적 방지 ① 과도 맞음
정지 회원은 보임	정지는 관리자 조치일 뿐, 협업 화면(Phase 6)의 다른 멤버에게까지 사진을 숨길 이유가 없다. 바꾸려면 조건 한 줄
모든 404 를 같은 문구로	어떤 이름이 "있었는데 탈퇴" 인지 "처음부터 없음" 인지 구분할 수 없게
소문자만 허용	`uuid4().hex` 는 늘 소문자다. DB 비교는 대소문자를 무시하지만, 주소는 한 가지 모양만 받아 같은 사진이 여러 주소로 열리지 않게
대문자 · 형식 오류도 404 (422 아님)	파일 주소이므로 "그런 파일 없음" 이 자연스럽다

알아 둘 점 — 색인: `profile_image_path` 칸에는 색인(index)이 없어 사진을 열 때마다 회원 표를 처음부터 훑는다. 회원이 수천 명까지는 문제없다. 많아지면 `users.profile_image_path` 에 UNIQUE 색인을 더하면 된다 (DB.md 변경 — 그때 사용자와 정함).

6. 시험 (`tests/test_user_service.py` 에 12개 추가 · 모두 56개, 실제 MySQL + 임시 폴더)

시험	확인한 것
활동 회원의 사진	storage 안 절대 경로, 파일 내용 같음
이상한 이름 7가지	`../../.env` · 인코딩된 `..%2F` · 대문자 · `.gif` · `.jpg.exe` · 31자 · 빈 글자 → 404
아무도 안 쓰는 이름	404
탈퇴 회원	404
정지 회원	경로를 돌려줌
디스크에 파일 없음	404

모든 404 는 `NOT_FOUND` · "Image not found." 로 같음을 확인. 전체 시험 290개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 41 — `GET · PATCH /api/v1/users/me`

항목	내용
파일	`studionx/backend/app/api/v1/users.py` — `read_me()` · `update_me()`
작업	P1-08 회원 정보 — 내 정보 조회 · 닉네임 변경 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	40 `profile_image_file()` · 42 `POST · DELETE /users/me/profile-image`

1. 하는 일

요청	응답
`GET /api/v1/users/me` + `Authorization: Bearer <Access>`	200 내 정보 7칸 (36)
`PATCH /api/v1/users/me` + `{"nickname": " Route_New "}`	200 바뀐 내 정보 (`"nickname": "Route_New"`)
`PATCH` + `{}`	200 그대로
`PATCH` + `{"email": "new@example.com"}`	200 그대로 — 이메일은 바꿀 수 없음 (FR-106), 모르는 칸은 무시
`PATCH` + `{"nickname": " a "}` · 21자	422 `VALIDATION_ERROR`, `details.nickname`
`PATCH` + 남이 쓰는 닉네임	409 `NICKNAME_TAKEN` · 탈퇴 3일 안이면 409 `REJOIN_TOO_SOON` (37)
토큰 없음 · 틀림 · 탈퇴 회원	401 `UNAUTHORIZED` (06 `CurrentUser`)

2. 언제 쓰나

화면 (JS)	부르는 것
userInfo.html `loadMe` (P1-10)	`GET` — 닉네임 · 이메일 · 사진 칸 채우기
userInfo.html 닉네임 Save changes (P1-10)	`PATCH`
myPage.html (P1-17)	대시보드 API(P1-16)가 따로 있음 — 이 API 는 회원 정보 화면용

함수들이 이어지는 모습 (`PATCH`)

순서	하는 곳	일
①	06 `CurrentUser`	토큰 → 활동 회원 (아니면 401)
②	`MeUpdateRequest` (37)	본문 검사 — 공백 제거 후 2~20자 (아니면 422, 03 처리기)
③	37 `update_me`	중복 · 탈퇴 3일 확인 → 저장 (아니면 409)
④	36 `to_me_response`	7칸 응답

3. 코드

import (파일 맨 위)

```
from fastapi import APIRouter, Response, UploadFile, status
from fastapi.responses import FileResponse

from app.api.deps import CurrentUser, Db
from app.schemas.common import ErrorResponse
from app.schemas.user import (
    MeResponse,
    MeUpdateRequest,
    PasswordChangeRequest,
    WithdrawRequest,
)
from app.services import user_service
```

`user_service` 줄을 이번에 추가했다.

함수 (경로 · 응답 설정은 템플릿 그대로)

```
@router.get("/me", response_model=MeResponse, responses={401: {"model": ErrorResponse}})
def read_me(user: CurrentUser) -> MeResponse:
    return user_service.to_me_response(user)


@router.patch(
    "/me",
    response_model=MeResponse,
    responses={409: {"model": ErrorResponse}, 422: {"model": ErrorResponse}},
)
def update_me(data: MeUpdateRequest, user: CurrentUser, db: Db) -> MeResponse:
    return user_service.to_me_response(user_service.update_me(db, user, data))
```

문법 풀이

import 문법	뜻
`from app.services import user_service`	모듈 통째로 — `user_service.update_me` 처럼 **어느 파일의 함수인지** 보이게 (라우터 함수 이름도 `update_me` 라 겹치지 않음)

식	뜻
`@router.get("/me", …)` · `@router.patch(…)`	이 라우터는 `/api/v1/users` 에 붙어 있음 → `GET · PATCH /api/v1/users/me`
`user: CurrentUser`	06 의 의존성 — 토큰을 검사해 회원을 넣어 줌
`data: MeUpdateRequest`	JSON 본문을 이 모양으로 검사해서 넣어 줌. 틀리면 함수가 불리지도 않고 422
`db: Db`	요청마다 DB 세션 (P1-03)
`response_model=MeResponse`	응답을 이 모양으로 한 번 더 걸러 보냄 · 문서(`/docs`)에 표시

4. 흐름

GET: 토큰 확인 → 36 으로 응답. **PATCH:** 토큰 확인 → 본문 검사 → 37 로 바꾸기 → 36 으로 응답.

5. 왜 이렇게

선택	이유
라우터는 한 줄 (일은 서비스에)	P1-04 · 05 와 같은 원칙 — 규칙은 서비스에서 시험하고, 라우터는 연결만
모르는 칸(`email`)은 무시	pydantic 기본 동작. 이메일을 바꾸려는 요청도 오류 없이 "안 바뀜" 으로 끝난다 — 화면은 이메일 칸을 읽기 전용으로 보여 줌
바뀐 정보를 응답으로	화면이 다시 `GET` 하지 않아도 된다
`user_id` 가 응답에 있어도 됨	토큰 주인 본인에게만 감 (36 §2)

6. 시험 (`tests/test_user_routes.py` 에 9개 추가 · 모두 11개, 실제 앱 + MySQL)

시험	확인한 것
`GET`	200, 7칸, 값, 사진 `null`, 응답에 비밀번호 해시 글자 없음
`GET` 토큰 없음	401 `UNAUTHORIZED`
`PATCH` 새 닉네임	200 `Route_New`, DB 도
`PATCH {}`	200 그대로
`PATCH` 이메일	200, 이메일 그대로
`PATCH " a "` · 21자	422 `VALIDATION_ERROR`, `details` 에 `nickname`
`PATCH` 남의 닉네임 (대소문자만 다름)	409 `NICKNAME_TAKEN`
`PATCH` 토큰 없음	401

전체 시험 299개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 42 — `POST · DELETE /api/v1/users/me/profile-image`

항목	내용
파일	`studionx/backend/app/api/v1/users.py` — `upload_profile_image()` · `delete_profile_image()`
작업	P1-08 회원 정보 — 프로필 사진 올리기 · 지우기 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	41 `GET · PATCH /users/me` · 43 `GET /profile-images/{name}`

1. 하는 일

요청	응답
`POST` multipart, 칸 `file` = JPG(1.2MB)	200 내 정보 — `"profile_image_url": "/api/v1/profile-images/3f9c….jpg"`
`POST` GIF	415 `UNSUPPORTED_FILE_TYPE`
`POST` 2MB 초과	413 `FILE_TOO_LARGE`
`POST` 칸 이름이 `file` 이 아님 (`photo`)	422 `VALIDATION_ERROR`
`DELETE`	200 내 정보 — `"profile_image_url": null`, 파일 삭제
`DELETE` (사진 없음 · 두 번)	200
토큰 없음	401 — 파일을 저장하지 않음

2. 언제 쓰나

화면 (userInfo.html, P1-10)	부르는 것
Upload photo → 파일 고르기	`FormData` 에 `file` 을 넣어 `api.post('/users/me/profile-image', formData)` → 응답의 `profile_image_url` 로 `<img>` 바꾸기
Remove	`api.delete('/users/me/profile-image')` → 첫 글자 동그라미로

함수들이 이어지는 모습 (`POST`)

순서	하는 곳	일
①	06 `CurrentUser`	토큰 → 활동 회원 (아니면 401, 파일은 읽지도 않음)
②	FastAPI `UploadFile`	multipart 의 `file` 칸 (없으면 422)
③	38 `save_profile_image`	종류 · 크기 · 내용 검사 → 저장 → 예전 파일 삭제
④	36 `to_me_response`	새 사진 주소(35)가 담긴 응답

3. 코드

import (파일 맨 위 — 41 과 같음, 추가 없음)

함수 (경로 · 응답 설정은 템플릿 그대로)

```
@router.post(
    "/me/profile-image",
    response_model=MeResponse,
    responses={413: {"model": ErrorResponse}, 415: {"model": ErrorResponse}},
)
async def upload_profile_image(file: UploadFile, user: CurrentUser, db: Db) -> MeResponse:
    user = await user_service.save_profile_image(db, user, file)
    return user_service.to_me_response(user)


@router.delete("/me/profile-image", response_model=MeResponse)
def delete_profile_image(user: CurrentUser, db: Db) -> MeResponse:
    return user_service.to_me_response(user_service.delete_profile_image(db, user))
```

문법 풀이

식	뜻
`file: UploadFile`	multipart 본문의 `file` 칸을 파일로 받음. 큰 파일은 디스크 임시 파일에 두어 메모리를 아낌. `python-multipart` 패키지가 필요 (requirements 에 있음)
`async def` · `await`	38 이 `await file.read(…)` 를 쓰므로 라우터도 비동기
`user = await …`	매개변수 `user` 에 저장 결과(같은 회원, 새 경로)를 다시 넣음

4. 흐름

POST: 토큰 확인 → `file` 칸 받기 → 38 로 검사 · 저장 → 36 응답. **DELETE:** 토큰 확인 → 39 로 지우기 → 36 응답.

5. 왜 이렇게

선택	이유
라우터는 연결만	검사 · 저장 순서 · 실패 처리는 38 · 39 에서 시험을 마쳤다
응답 = 내 정보 전체	화면이 새 사진 주소를 바로 받아 `<img src>` 를 바꾼다 (다시 `GET` 불필요)
사진이 없어도 `DELETE` 200	39 와 같음 — 두 번 눌러도 사용자에게 오류를 보이지 않는다
토큰 검사가 파일보다 먼저	로그인하지 않은 사람이 파일을 올려도 저장되지 않는다 (시험으로 확인: `profiles` 폴더가 생기지도 않음)

알아 둘 점 — 비동기 안의 DB: `upload_profile_image` 는 `async` 인데, 38 안의 DB 저장(`commit`)은 동기 방식이라 그동안 서버의 다른 요청이 아주 잠깐 기다린다. 사진 저장은 한 번에 수 밀리초라 Phase 1 에서는 문제없다. 사용자가 많아지면 파일만 먼저 읽고 DB 일은 동기 함수로 나누면 된다.

6. 시험 (`tests/test_user_routes.py` 에 7개 추가 · 모두 18개, 실제 앱 + MySQL + 임시 폴더)

시험	확인한 것
JPG 올리기	200, 주소 모양 `/api/v1/profile-images/{32자}.jpg`, 그 파일 내용이 올린 것과 같음
GIF · 2MB 초과	415 `UNSUPPORTED_FILE_TYPE` · 413 `FILE_TOO_LARGE`
칸 이름 `photo`	422 `VALIDATION_ERROR`
토큰 없이 올리기	401, 저장 폴더가 생기지도 않음
지우기	200, 주소 `null`, 파일 삭제 → 한 번 더 지우기도 200
토큰 없이 지우기	401

전체 시험 306개 통과, `ruff` 검사 통과. 진짜 `storage/` 폴더는 비어 있음(`.gitkeep` 만).

Backend(FastAPI/Python) 43 — `GET /api/v1/profile-images/{name}`

항목	내용
파일	`studionx/backend/app/api/v1/users.py` — `images_router` · `read_profile_image()` / `app/api/v1/router.py` (한 줄)
작업	P1-08 회원 정보 — 프로필 사진 파일 보내기 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	42 `POST · DELETE /users/me/profile-image` · (P1-08 마지막)

1. 하는 일

`<img src="/api/v1/profile-images/3f9c….jpg">` 가 부르는 주소. 사진 **파일 자체**를 보낸다. 로그인은 필요 없다.

요청	응답
올린 사진의 주소 (토큰 없이)	200 그림 바이트 · `Content-Type: image/jpeg` (png · webp 도 맞게) · `X-Content-Type-Options: nosniff` · `Cache-Control: public, max-age=86400`
없는 이름 · 모양이 아닌 이름	404 `NOT_FOUND` "Image not found."
지운 사진 · 탈퇴한 회원의 사진	404 (같음)
`..%2F..%2F.env`	404 — `/` 로 풀려 이 주소와 맞지도 않음 (앱 공통 404)
템플릿의 `/api/v1/users/85/profile-image`	404 — **없는 주소**

2. 언제 쓰나

쓰는 곳	
화면의 `<img src>`	userInfo.html · myPage.html 의 프로필 사진 (P1-10 · P1-17), 협업 멤버 사진 (Phase 6)
주소는 어디서	36 응답의 `profile_image_url` (35 가 만듦) — 화면은 받은 주소를 그대로 넣기만

사용자 결정 반영 (35 §2)

템플릿은 `GET /users/{user_id}/profile-image` 였다. **회원 번호로 여는 공개 주소를 만들지 않는다**(탈퇴 추적 방지 ③)는 결정에 따라 `/profile-images/{무작위 이름}` 으로 바꿨다. 이 주소는 `/users` 아래가 아니어서 라우터를 하나 더 두었다.

바꾼 곳	내용
`users.py`	`images_router = APIRouter()` 와 그 위의 라우터 함수
`router.py`	`api_router.include_router(users.images_router, prefix="/profile-images", tags=["users"])`
`tests/test_user_routes.py`	연결 확인 시험을 새 주소로, 옛 주소는 **없음**을 확인

3. 코드

import (파일 맨 위 — 41 과 같음, 추가 없음). 이 함수는 `FileResponse` 와 40 `profile_image_file` 을 쓴다.

라우터 · 상수 (이번에 추가)

```
# 사진 파일 — /api/v1/profile-images/{name} (회원 번호가 아니라 무작위 파일 이름, 2026-10-07)
# /users 아래가 아니라서 라우터를 따로 둠 → router.py 에서 prefix="/profile-images" 로 붙임
images_router = APIRouter()

# 확장자 → 응답 종류 (PHOTO_TYPES 를 뒤집음: ".jpg" → "image/jpeg")
PHOTO_MEDIA_TYPES = {ext: media for media, ext in user_service.PHOTO_TYPES.items()}
```

함수

```
@images_router.get(
    "/{name}",
    response_class=FileResponse,
    responses={404: {"model": ErrorResponse}},
)
def read_profile_image(name: str, db: Db) -> FileResponse:
    path = user_service.profile_image_file(db, name)
    return FileResponse(
        path,
        media_type=PHOTO_MEDIA_TYPES[path.suffix],
        headers={"Cache-Control": "public, max-age=86400", "X-Content-Type-Options": "nosniff"},
    )
```

문법 풀이

식	뜻
`{ext: media for media, ext in ….items()}`	**사전 만들기 식** — `PHOTO_TYPES`(`"image/jpeg": ".jpg"`)의 키와 값을 뒤집은 사전
`"/{name}"` · `name: str`	주소의 마지막 부분이 `name` 매개변수로 들어옴
`FileResponse(path, …)`	파일을 조금씩 읽어 보냄 (통째로 메모리에 올리지 않음)
`path.suffix`	확장자 `".jpg"`
`response_class=FileResponse`	문서(`/docs`)에 "파일 응답" 으로 표시

4. 흐름

1. 40 으로 파일을 찾는다 — 못 보여 주면 거기서 404.
2. 확장자로 종류를 정하고, 캐시 · `nosniff` 헤더를 붙여 파일을 보낸다.

5. 왜 이렇게

선택	이유
로그인 확인 없음	`<img>` 는 `Authorization` 헤더를 붙일 수 없다. 대신 주소가 무작위 32자라 이름을 모르면 열 수 없다 (2^{128} 가지)

선택	이유
종류를 **확장자로** 직접 정함	38 이 내용을 보고 확장자를 붙였으니 확장자 = 실제 종류. 운영체제의 종류 표에 기대지 않는다 (Windows 는 `.webp` 를 모를 수 있음)
`X-Content-Type-Options: nosniff`	브라우저가 내용을 보고 "사실은 HTML 이네" 하고 바꿔 해석하지 않게 — 38 의 내용 검사와 이중 잠금
캐시 1일 (`max-age=86400`, 템플릿 제안)	같은 사진을 매번 받지 않는다. 사진을 바꾸면 **이름이 바뀌어** 새로 받는다. 더 길게(1년) 할 수도 있지만, 지운 사진 · 탈퇴한 회원의 사진이 다른 사람 브라우저에 오래 남지 않게 하루로
`public`	사진은 이름만 알면 누구나 보는 자료 — 공유 캐시(CDN)에 둬도 같다
라우터를 `users.py` 안에	회원 사진은 회원 정보의 일부. 파일을 새로 만들면 템플릿 머리말 · 안내가 갈라진다

6. 시험 (`tests/test_user_routes.py` 에 8개 추가 · 모두 26개, 실제 앱 + MySQL + 임시 폴더)

시험	확인한 것
jpg · png · webp 올린 뒤 응답 주소로 열기 (토큰 없이)	200, 바이트가 같음, `Content-Type` 이 각각 맞음, `nosniff`, 캐시 헤더
없는 이름 · `..%2F..%2F.env` · 모양 아님	404 `NOT_FOUND`
지운 사진 · 탈퇴 회원의 사진	404
옛 주소 `/users/{번호}/profile-image`	열리지 않음
연결 확인 (DB 없이)	`/api/v1/profile-images/{name}` 있음, 옛 주소 없음

처음 1개 실패: `%2F` 는 `/` 로 풀려 사진 주소와 맞지 않아 이 함수까지 오지 않고 앱 공통 404("Not found.")가 났다. 안전한 결과라 그 경우는 상태 · 코드만 확인하도록 시험을 고쳤다. 전체 시험 314개 통과, `ruff` 검사 통과. 진짜 `storage/` 는 비어 있음.